Mini-project 1 in Advanced Machine Learning (02460)

by Carl N. (s214624), Botong Y. (s214623), Malthe B. (s214631) & Jacob N. (s204093) on 05 March 2026

Part A: Priors for VAEs

In this section, we trained VAEs [2] on binarized MNIST [3]. Before comparisons between priors, we parameter-swept for each VAE/prior combination. We created a validation set from the training set, using a 10/90 split. We show the selected parameters/results in Table 1.

Table 1: Hyperparameters and results.

Prior	Latent Dim	N	Test result
Gaussian	64	-	-81.7577 ± 0.438
MoG	32	7	-80.837 ± 0.469
Flow	16	32	-76.5902 ± 0.349

N indicates components for MoG, N-transforms for flow. The values in the *Test result* column are the negative log-likelihood scores on the test set, approximated using the ELBO. We ran all experiments with a constant batch-size of 256 and a learning rate of 0.001. We used zero mean and unit variance for Gaussian, diagonal covariance-matrices for MoG and alternating masking for Flow. The test log-likelihood scores are similar for the Gaussian and MoG priors with the MoG prior edging out, whereas the model with a Flow prior performs substantially better. Overall, they show a pattern of more complicated priors providing better results. The MoG and Flow are both fitted to the data, providing a clear advantage, but they are also more complex in nature, allowing them to be more informative. We also observe an interesting pattern, that complex priors give rise to smaller latent dimension. Illustrations of the priors and aggregate posteriors can be seen in Figure 1. These are from variants trained with a 2D latent dimension for visualisation purposes. We can clearly see how the prior influences the aggregate posterior, particularly as the prior gets more informative.

We used a Gaussian encoder and a Bernoulli decoder. The Gaussian encoder is standard practice, but the Bernoulli decoder is used as we're trying to reconstruct binarized data.

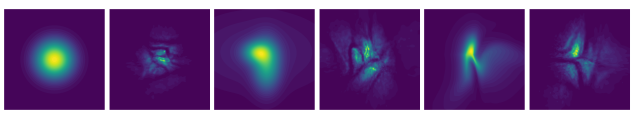

Figure 1: Left to right: prior / aggregate posterior of Gaussian, MoG, Flow

Part B: Sampling quality of generative models

We implemented two diffusion-based generative models. A U-Net DDPM [1] (SiLU activation) was trained directly in pixel space to predict the added Gaussian noise, while a latent diffusion model used a β-VAE with Gaussian likelihood to learn a latent representation. The β-VAE was evaluated with three different values of the KL-weighting parameter β, and the models were trained for 40 epochs with a batch size of 64.

Figure 2: Generated samples from the different models. From left to right: L-DDPM with $\beta = 10^{-6}$, L-DDPM with $\beta = 1$, L-DDPM with $\beta = 2$, β-VAE with $\beta = 2$, and the U-Net DDPM model.

Table 2: FID and sampling time

Model	U-Net DDPM	L-DDPM B=1	L-DDPM B=1e-6	L-DDPM B=2	B-VAE B=2
FID	25.96	48.64	74.06	44.68	95.32
Wall clock Sampling time Samples/sec	1.35	252.6	253.9	253.1	45.045

The results show a clear trade-off between sample quality and sampling efficiency across the models. The U-Net DDPM achieves the best generative quality with an FID of 25.97, but sampling is extremely slow at 1.35 samples/sec. In contrast, the latent β-VAE + DDPM models generate samples much faster while maintaining moderate sample quality, with the best configuration ($\beta = 2$) reaching an FID of 44.68. The β-VAE alone is by far the fastest (45,045 samples/sec) since it requires only a single forward pass through the decoder, but this comes at the cost of poor sample quality (FID 95.32). Overall, the results show that latent diffusion significantly improves sampling efficiency compared to pixel-space diffusion while achieving substantially better quality than a standalone VAE.

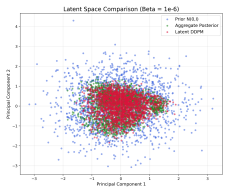

Figure 3: Latent DDPM samples.

The prior $p(z) = \mathcal{N}(0, I)$ (blue) is much more spread out than the aggregate posterior $q(z)$ (green), showing that the encoder does not match the Gaussian prior when $\beta = 10^{-6}$. The latent DDPM samples (red) closely follow the aggregate posterior instead of the prior, indicating that the diffusion model learns the true latent distribution even with a poorly performing β. This explains why latent DDPM sampling produces better FID scores than sampling directly from the VAE prior.

Code snippets

Here is the link to our github repo: https://github.com/carlLGN/02460Miniprojects
Code snippet for ELBO for non-gaussian:

```
kl = (q.log_prob(z) - self.prior().log_prob(z)).mean(0)
elbo = torch.mean(self.decoder(z).log_prob(x) - kl, dim = 0)
```

Flow Implementation:
This code calls the flow implementation from the group exercises. ... indicates something left out for brevity.

```
class FlowPrior(nn.Module):
    def __init__(self, M, n_transformations, num_hidden=32):
        ...
        mask[M//2:] = 1
        for _ in range(n_transformations):
            mask = (1-mask) #alternate

            scale_net = ...
            translation_net = ...
            ...
            transformations.append(MaskedCouplingLayer(scale_net, translation_net, mask))

        self.flow = Flow(base=base, transformations=transformations)
```

This code implements the DDPM negative elbow on a batch of data.

```
t = torch.randint(0, self.T, (batch_size,), device=device).long()
epsilon = torch.randn_like(x)
alpha_bar_t = self.alpha_cumprod[t].view(batch_size, *([1] * (len(x.shape) - 1)))

n_x = torch.sqrt(alpha_bar_t) * x + torch.sqrt(1 - alpha_bar_t) * epsilon
epsilon_theta = self.network(n_x, (t.float() / (self.T - 1)).unsqueeze(1))

neg_elbo = F.mse_loss(epsilon_theta, epsilon, reduction='none').flatten(start_dim=1).sum(dim
    =-1)
```

This code implements the DDPM sampling algorithm.

```
x_t = torch.randn(shape).to(self.alpha.device)
for t in range(self.T-1, -1, -1):
    ### Implement the remaining of Algorithm 2 here ###
    if t > 0: #t=1 in the book but t=0 here because of indexing
        z = torch.randn_like(x_t)
    else:
        z = torch.zeros_like(x_t)
    alpha_bar_t = self.alpha_cumprod[t].view(1,*([1]* (len(shape)- 1)))
    t_in = torch.full((x_t.shape[0], 1), t, device=self.alpha.device, dtype=torch.float)/(self
    .T - 1)

    epsilon_theta = self.network(x_t, t_in)
    x_t = (x_t-(1 -self.alpha[t])/ torch.sqrt(1 - alpha_bar_t) *epsilon_theta) /torch.sqrt(
    self.alpha[t]) + torch.sqrt(self.beta[t]) * z
```

Central parts of the Latent DDPM training and sampling steps. Training (latent DDPM on VAE latents)

```
q = vae.encoder(x)
z = q.rsample()
loss = ddpm_latent.loss(z)
loss.backward()
optimizer.step()
```

Sampling (latent DDPM then decode with VAE)

```
z = latent_model.sample((args.n_samples, M))
px = vae.decoder(z)
x = px.mean
x_save = ((x.unsqueeze(1) + 1) / 2).clamp(0, 1)
```

Declaration of use of generative AI

This declaration **must** be filled out and included as the 3rd page of the document. The questions apply to all parts of the work, including research, project writing, and coding.

- I/we have used generative AI tools: yes

If you answered *yes*, please complete the following sections. List the generative AI tools you have used:

- Google Gemini
- Claude

Describe how the tools were used:

What did you use the tool(s) for? We used the tools for understanding, plotting, debugging and coding.

At what stage(s) of the process did you use the tool(s)? Through the entire process.

How did you use or incorporate the generated output? Mostly to deepen our understanding and to work out solutions. Individual times, we've used code directly, but it is noted at the script.

References

[1] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[2] D. P. Kingma and M. Welling. Auto-encoding variational bayes. *International Conference on Learning Representations (ICLR)*, 2014.

[3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.